



UNIVERSITÀ
degli STUDI
di CATANIA

DIPARTIMENTO DI
INGEGNERIA ELETTRICA ELETTRONICA ED INFORMATICA
CORSO DI LAUREA MAGISTRALE IN INGEGNERIA INFORMATICA

RELAZIONE TECNICA APL

DEEP PACKET INSPECTOR -APPLICAZIONE A MICROSERVIZI PER L'ANALISI
DEI PACCHETTI IN RETE PROTETTA

Alunni:

Arena Giorgia
Tornabene Alessio

Docenti:

Vincenza Carchiolo
Marco Grassia

ANNO ACCADEMICO 2025 – 2026

INDICE

- 1. Introduzione e Obiettivi del Progetto3
- 2. Architettura Distribuita e Comunicazione (IPC)5
- 3. Il Sensore di Cattura (C++)9
- 4. Orchestrazione Concorrente e Web Server (Go)..... 10
- 5. Analisi Forense (Python)..... 15
- 6. Testing Deterministico: Il Generatore di Traffico..... 17
- 7. Orchestrazione, Sicurezza e Deploy (Docker Compose)..... 19
- 8. Conclusioni..... 22

1. Introduzione e Obiettivi del Progetto

1.1 Descrizione del Dominio: Il Deep Packet Inspection (DPI) e i limiti degli approcci monolitici

Nel panorama odierno della sicurezza informatica e dell'ingegneria delle reti, il Deep Packet Inspection (DPI) rappresenta una tecnologia fondamentale per il monitoraggio e la protezione delle infrastrutture. A differenza dei firewall tradizionali (Stateful Packet Inspection), che limitano la loro indagine all'intestazione dei pacchetti (livelli 3 e 4 del modello OSI, come IP e TCP), un sistema DPI si spinge fino al livello applicativo (Livello 7), analizzando il *payload* vero e proprio alla ricerca di anomalie, malware, fughe di dati o traffico non conforme.

Storicamente, i sistemi DPI sono stati progettati secondo un paradigma monolitico, in cui un singolo enorme processo si fa carico dell'intero ciclo di vita del dato: dall'intercettazione fisica sulla scheda di rete, al parsing degli header, fino alla complessa elaborazione testuale per la ricerca delle firme virali (pattern matching). Questo approccio presenta un limite ingegneristico severo, ovvero la creazione di colli di bottiglia computazionali. La cattura dei pacchetti richiede tempi di reazione dell'ordine dei microsecondi e deve interfacciarsi a bassissimo livello con il kernel del sistema operativo; al contrario, l'ispezione testuale tramite espressioni regolari (Regex) è un'operazione *CPU-bound* intrinsecamente lenta. In un'architettura monolitica, un picco di traffico o un payload particolarmente complesso da ispezionare monopolizza le risorse del processo centrale, impedendo la lettura tempestiva della scheda di rete e causando l'inevitabile perdita di pacchetti (packet loss), vanificando così l'affidabilità dell'intero sistema di sicurezza.

1.2 Obiettivi del Progetto

L'obiettivo primario di questo progetto è superare i vincoli imposti dalle architetture monolitiche sviluppando un sistema distribuito per l'analisi in tempo reale del traffico di rete grezzo. Il sistema è stato ingegnerizzato per garantire latenza minima e massima

scalabilità, realizzando una separazione netta tra il dominio del *networking* e quello della *computazione analitica*.

Le funzionalità principali previste dal progetto includono la cattura silenziosa dei pacchetti a basso livello, la decodifica strutturata degli header di trasporto e l'ispezione profonda dei payload alla ricerca di pattern sospetti (come credenziali trasmesse in chiaro o firme di attacchi noti). Oltre all'elaborazione backend, l'architettura si pone l'obiettivo di fornire visibilità immediata all'operatore di sicurezza, esponendo una dashboard web interattiva alimentata in tempo reale tramite protocollo WebSocket. Infine, per garantire un'orchestrazione immediata e una riproducibilità totale dell'ambiente di esecuzione, l'intera infrastruttura è stata containerizzata sfruttando Docker Compose.

1.3 La Scelta Architetture: Paradigma a Microservizi e Integrazione Poliglotta

Per far fronte alle sfide di un sistema DPI moderno, l'applicazione non è stata concepita come un singolo programma, ma segue un'architettura modulare distribuita a tre livelli. Invece di forzare l'uso di un singolo linguaggio di programmazione compromettendone l'efficienza globale, si è optato per un paradigma a microservizi poliglotta, implementando una rigorosa pipeline di filtraggio in cui ogni nodo è sviluppato nel linguaggio che ne massimizza le prestazioni specifiche:

- **Cattura (C++):** Il primo modulo funge da sonda (Packet Sniffer). Interagendo a bassissimo livello con il sistema operativo, è responsabile esclusivo della cattura del traffico di rete grezzo. L'adozione del C++ garantisce un *footprint* di memoria minimo e un'esecuzione multi-threaded capace di intercettare i frame di rete con latenza quasi nulla.
- **Orchestrazione e Routing (Go):** Il cuore dell'architettura è un router concorrente ad altissime prestazioni. Il linguaggio Go è stato scelto per le sue primitive native di concorrenza (Goroutine e canali). Questo modulo riceve il flusso massivo di byte dal sensore, è capace di assorbire i picchi di traffico, e si occupa della decodifica parallela degli header IP e TCP. Scartato il traffico

irrilevante, Go funge anche da Web Server concorrente, gestendo il flusso degli alert verso la Dashboard Web tramite WebSockets.

- **Analisi (Python):** Il terzo livello è isolato dal caos della rete fisica ed è dedicato esclusivamente all'ispezione logica. Python è stato selezionato per la sua insuperabile espressività nella manipolazione di stringhe e nell'analisi dati avanzata. Questo servizio riceve unicamente i payload rilevanti pre-filtrati dal router, si occupa del delicato riassettaggio delle sessioni TCP ed esegue il vero e proprio *pattern matching* tramite Espressioni Regolari (Regex), generando infine i log formali delle anomalie.

2. Architettura Distribuita e Comunicazione (IPC)

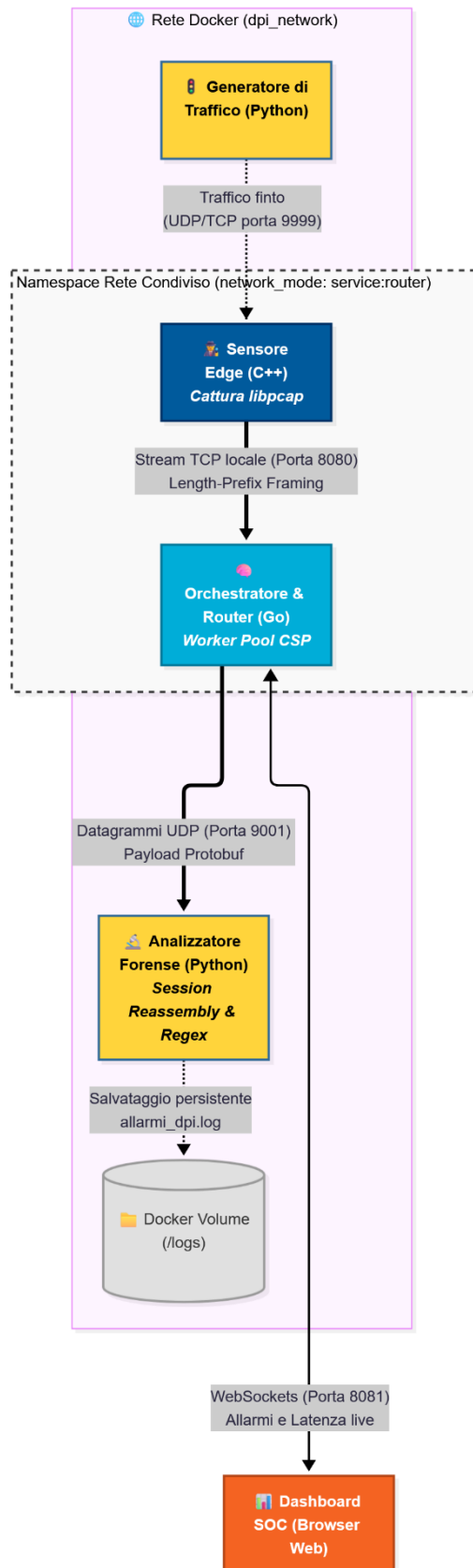
2.1 La Pipeline a Tre Stadi

Il cuore dell'applicazione è strutturato su una pipeline di filtraggio direzionale composta da tre nodi, ciascuno delegato a una fase specifica del ciclo di vita del dato. Il flusso delle informazioni è rigorosamente unidirezionale per evitare colli di bottiglia e dipendenze cicliche:

1. **Sensore di cattura (C++):** Agisce in prima linea catturando il traffico grezzo direttamente dall'interfaccia di rete e iniettandolo nel sistema ad altissima velocità.
2. **Orchestratore (Go):** Riceve il flusso grezzo dal sensore, esegue la decodifica e lo smistamento dei pacchetti in modo concorrente, scartando il rumore di fondo (traffico benigno di tipo HTTP/FTP) ed esponendo parallelamente una Dashboard Web in tempo reale.
3. **Analizzatore (Python):** Riceve dall'orchestratore esclusivamente i payload di interesse, procedendo alla ricostruzione delle sessioni TCP e all'applicazione delle regole di *pattern matching* (Regex) per la generazione degli allarmi.

Progettare un sistema a microservizi significa affrontare la sfida intrinseca della comunicazione tra processi indipendenti, nota come IPC (*Inter-Process Communication*). Per garantire la massima efficienza, i protocolli di trasporto sono stati

scelti in base alle specifiche esigenze di ciascuna tratta. In seguito una figura che descrive meglio il comportamento del Dpi:



2.2 Inter-Process Communication: TCP e il pattern Length-Prefix Framing

Per la comunicazione critica tra il Sensore C++ e l'Orchestratore Go, la scelta è ricaduta su uno stream TCP locale. In questa fase iniziale, la perdita anche di un singolo byte corromperebbe irreparabilmente i frame di rete grezzi, rendendo imperativa l'affidabilità garantita dal protocollo TCP.

Tuttavia, il TCP è un protocollo orientato allo stream, intrinsecamente privo del concetto di "confine del messaggio" (*message boundary*). Se il C++ inviasse più frame in rapida successione, il router Go rischierebbe di leggerli fusi in un unico blocco continuo, causando errori fatali di spaccettamento. Per risolvere questa problematica a livello architetturale, è stato implementato il design pattern **Length-Prefix Framing**. Prima di inviare il payload effettivo, il modulo C++ antepone un'intestazione fissa di 4 byte contenente la dimensione esatta del pacchetto in arrivo. Il router Go, di conseguenza, legge prima i 4 byte, deduce la lunghezza esatta del frame, e attende di aver riempito il buffer unicamente per quella precisa quantità di dati. Questo approccio garantisce una sincronizzazione perfetta e previene la frammentazione applicativa.

2.3 L'efficienza del protocollo UDP

Superato il collo di bottiglia della cattura iniziale, la comunicazione tra il router Go e l'analizzatore Python cambia radicalmente paradigma, passando all'utilizzo di datagrammi UDP. Poiché questa trasmissione avviene esclusivamente all'interno della rete virtuale isolata e controllata di Docker (*dpi_network*), il rischio di congestione fisica o perdita di pacchetti tipica delle reti WAN è nullo. L'adozione del protocollo UDP elimina l'overhead computazionale del *three-way handshake* e l'attesa degli *acknowledgement* (ACK) del TCP. Questo garantisce un *throughput* massimale e una latenza di inoltro ridotta al minimo indispensabile, permettendo al modulo Python di ricevere i frammenti TCP estratti dal router in tempo quasi reale.

2.4 Serializzazione dei dati: Protocol Buffers (Protobuf)

L'utilizzo di un'architettura poliglotta solleva il problema dell'interoperabilità dei tipi di dato complessi (interi, stringhe, array di byte) tra C++, Go e Python. L'impiego di

formati testuali standard come JSON o XML è stato scartato a causa dell'eccessivo peso in termini di larghezza di banda e dell'oneroso tempo di parsing CPU-bound.

Per garantire uno scambio di informazioni ultraleggero, il sistema adotta i **Protocol Buffers (Protobuf)** di Google. Protobuf si basa su un contratto esplicito (il file .proto) che definisce rigorosamente la struttura del pacchetto, serializzando i dati in un formato binario altamente compresso.

```
syntax = "proto3";
package packet_inspector;
option go_package = "./router;router";

message NetworkPacket {

    //campi riempiti dal modulo Go
    string source_ip = 1;
    string dest_ip = 2;
    int32 source_port = 3;
    int32 dest_port = 4;
    string protocol = 5;      // Es: "TCP", "UDP"
    bytes raw_payload = 6;    //payload dei dati
    int64 timestamp_us = 7;   // Timestamp iniettato in C++ (Hardware) per calcolare la latenza end-to-end
    uint32 seq_num = 8;       // Sequence Number TCP per riordinare i pacchetti
    uint32 tcp_flags = 9;     // Flag grezzi del TCP (FIN, SYN, RST...) estratti da Go per l'analisi in Python
}
```

Questo garantisce:

- **Sicurezza dei tipi (Type Safety):** I tipi di dato (come gli *Uint32* per i Sequence Number) mantengono la loro integrità matematica viaggiando tra linguaggi con regole di tipizzazione differenti.
- **Velocità e Leggerezza:** La serializzazione binaria occupa una frazione dello spazio rispetto all'equivalente JSON, riducendo drasticamente il carico sulla rete interna di Docker.
- **Resilienza:** In fase di deserializzazione nel modulo Python, eventuali anomalie o corruzioni del datagramma vengono intercettate elegantemente tramite la cattura dell'eccezione nativa *DecodeError*, garantendo che l'analizzatore scarti il pacchetto errato senza causare il crash dell'intero microservizio.

3. Il Sensore di Cattura (C++)

3.1 L'Acquisizione a Basso Livello

Il primo stadio della pipeline è rappresentato dal Sensore di cattura, sviluppato in linguaggio C++ per garantire un'interazione a bassissimo livello con il kernel del sistema operativo e minimizzare l'overhead di memoria. Il modulo si basa sulla libreria standard **libpcap**, che permette di mettere l'interfaccia di rete in modalità promiscua e intercettare i *frame* a Livello 2 (Datalink).

In un sistema DPI monolitico, la sonda si occuperebbe sia della cattura che della decodifica degli header (Ethernet, IP, TCP/UDP). Nel nostro approccio a microservizi, invece, il compito del sensore C++ è stato intenzionalmente limitato alla sola acquisizione: legge il blocco di byte grezzi in transito sulla scheda di rete, appone un *timestamp* hardware ad altissima precisione (misurato in microsecondi) e lo immette immediatamente nel sistema. Delegando il gravoso compito del *parsing* degli header al livello successivo (l'Orchestratore Go), il sensore C++ mantiene un *footprint* di memoria irrisorio e previene il blocco del thread di cattura, scongiurando il rischio di perdita di pacchetti (packet loss) anche in presenza di throughput massivi.

Per ottimizzare ulteriormente le prestazioni e prevenire il cosiddetto "effetto eco" all'interno della rete virtuale Docker, il sensore è stato equipaggiato con un filtro BPF (Berkeley Packet Filter). Questo filtro scarta a monte il traffico locale generato dalla comunicazione intra-microservizio (es. le porte 8080, 8081 e 9001), permettendo a libpcap di concentrarsi esclusivamente sul traffico malevolo e legittimo iniettato dal generatore.

3.2 Concorrenza e Gestione della Memoria (Producer-Consumer)

Per isolare la latenza di rete dalla velocità di cattura hardware, l'architettura del sensore è stata progettata seguendo il classico paradigma di concorrenza **Producer-Consumer**.

Il sistema si compone di tre elementi fondamentali:

1. **CatturaTraffico (Producer):** Un thread dedicato esclusivamente all'ascolto continuo della scheda di rete e all'inserimento dei frame catturati in una struttura dati condivisa.
2. **CodaPacchetti (La Coda Condivisa):** Un'area di transito sicura (*thread-safe*) implementata tramite `std::queue`. L'integrità dei dati è garantita dall'uso combinato di semafori `std::mutex` e `std::condition_variable`. Quest'ultimo, in particolare, permette di "addormentare" il consumer quando la coda è vuota, evitando che sprechi cicli CPU e risvegliandolo istantaneamente tramite una `notify_one()` solo all'arrivo di un nuovo pacchetto.
3. **InoltroTraffico (Consumer):** Un thread indipendente che estrae i pacchetti dalla coda, li serializza in formato Protobuf e li trasmette al router Go tramite un socket TCP.

Per garantire l'efficienza assoluta nella gestione della memoria (prevenendo i classici *memory leak* del C++) senza gravare sul *Garbage Collector* inesistente nel linguaggio, si è fatto un uso massiccio degli *Smart Pointers* (`std::unique_ptr`) introdotti nel C++ moderno. Quando un pacchetto viene catturato, viene allocato dinamicamente in uno `unique_ptr`. Questo puntatore garantisce la "proprietà esclusiva" del dato: il pacchetto non viene mai copiato, ma viene letteralmente "spostato" (`std::move`) dal Producer alla Coda, e successivamente dalla Coda al Consumer. Nel momento in cui il Consumer invia il pacchetto sulla rete e l'oggetto esce dal suo *scope* (ciclo di vita), il distruttore dello *Smart Pointer* viene invocato automaticamente, liberando istantaneamente la memoria RAM e garantendo una stabilità architetturale di livello *enterprise*.

4. Orchestrazione Concorrente e Web Server (Go)

4.1 Il Modello CSP

Il secondo stadio della pipeline rappresenta il motore computazionale dell'intera architettura. Per il suo sviluppo è stato adottato il linguaggio Go, scelto specificamente per il suo runtime ottimizzato per la programmazione concorrente e i sistemi distribuiti. A differenza dei linguaggi tradizionali che si affidano ai pesanti thread del sistema

operativo e a complessi meccanismi di blocco della memoria condivisa (Mutex), Go implementa nativamente il modello matematico **CSP (Communicating Sequential Processes)**.

Nel modello CSP, i processi non comunicano condividendo la memoria, ma condividono la memoria comunicando. Questo paradigma si concretizza tramite l'uso delle **Goroutine** e dei **Canali**, condotti tipizzati e *thread-safe* attraverso i quali i processi si scambiano i dati. Questa scelta architetturale ha permesso di risolvere elegantemente il problema della **Backpressure** (contropressione). Il traffico grezzo proveniente dal sensore C++ viene iniettato all'interno di un canale *bufferizzato*. Questo canale funge da ammortizzatore: assorbe le micro-raffiche di traffico (spikes) senza bloccare il thread di cattura a monte. Qualora il volume di pacchetti in ingresso dovesse superare temporaneamente la capacità di elaborazione, la saturazione del buffer del canale induce un rallentamento controllato e prevedibile del *producer*, evitando catastrofici esaurimenti di memoria (Out-Of-Memory) e mantenendo il sistema in uno stato di massima stabilità.

4.2 Architettura a Worker Pool e Spacchettamento Bitwise ad Alte Prestazioni

Per lo smaltimento e l'elaborazione dei pacchetti accodati nel canale, il modulo implementa il *design pattern* **Worker Pool**. All'avvio del servizio, l'orchestratore istanzia un pool predefinito e limitato di Goroutine "lavoratrici". Questo limite fisso previene l'eccessivo overhead dovuto al continuo *context-switching* tra troppi thread, garantendo che il parallelismo sia perfettamente bilanciato con il numero di core logici effettivamente disponibili sull'hardware sottostante.

Ogni worker nel pool ha il compito di eseguire l'estrazione e la decodifica degli header di rete, il *parsing* dei protocolli IP e TCP avviene interamente in Go. Per massimizzare le prestazioni ed evitare librerie di alto livello che rallenterebbero il *throughput*, l'analisi è stata implementata tramite **operazioni Bitwise (manipolazione diretta a livello di bit)** sull'array di byte del payload grezzo. Lo spacchettamento procede matematicamente:

- **Header IPv4:** Per determinare la lunghezza variabile dell'intestazione IP, il worker isola i 4 bit meno significativi del byte iniziale tramite un operatore logico AND (byte & 0x0F) e moltiplica il valore decimale per 4, ottenendo l'offset esatto in cui inizia il protocollo di trasporto.
- **Porte e Payload TCP:** Le porte di origine e destinazione (campi a 16 bit) vengono estratte fondendo due byte adiacenti tramite lo scorrimento dei bit verso sinistra (*Bitwise Shift Left*, << 8) combinato con un operatore logico OR (|).
- **Analisi di Stato:** Con tecniche speculari di masking, il sistema estrae i **Sequence Number** a 32 bit (informazione vitale per il successivo riassettaggio dei payload frammentati) e i TCP Flags (SYN, ACK, PSH, FIN, RST), necessari per comprendere lo stato della connessione.

Terminato lo spaccettamento, i dati e i metadati rilevanti vengono serializzati in formato Protobuf e inoltrati all'analizzatore Python tramite protocollo UDP.

L'efficacia di questo spaccettamento a basso livello risulta evidente quando si analizza il ciclo di vita di una transazione di rete gestita dal router. L'estrazione mirata dei flag TCP tramite maschere di bit consente, infatti, all'orchestratore Go di identificare con precisione lo stato di avanzamento di ogni singola connessione.

The screenshot displays a network traffic analysis interface. On the left, a list of TCP connections is shown with columns for time, flag, source IP:port, and destination IP:port. On the right, detailed analysis logs are displayed for specific connections.

Time	Flag	Source IP:Port	Destination IP:Port
18:04:17.537	ACK	172.19.0.4:39612	172.19.0.3:9999
18:04:17.525	FIN	172.19.0.3:9999	172.19.0.4:39612
18:04:17.537	FIN	172.19.0.4:39612	172.19.0.3:9999
18:04:17.526	ACK	172.19.0.3:9999	172.19.0.4:39612
18:04:17.525	PSH	172.19.0.4:39612	172.19.0.3:9999
18:04:17.525	ACK	172.19.0.4:39612	172.19.0.3:9999
18:04:17.525	SYN	172.19.0.3:9999	172.19.0.4:39612
18:04:17.525	SYN	172.19.0.4:39612	172.19.0.3:9999

Analysis Log 1 (18:04:17.537):

```
/api/login HTTP/1.1 Host: router Content-Type: application/json
{"username": "admin", "password": "123456"}
```

Analisi completata in 380.32 ms

Analysis Log 2 (18:04:17.537):

CRITICAL SRC: 172.19.0.4:39612

> [ALERT TCP] Dati sensibili in chiaro rilevati | Payload: USER admin
PASS password

Analisi completata in 585.53 ms

Analysis Log 3 (18:04:15.523):

CRITICAL SRC: 172.19.0.4:39588

> [ALERT TCP] Dati sensibili in chiaro rilevati | Payload: POST
/api/login HTTP/1.1 Host: router Content-Type: application/json
{"username": "admin", "password": "root"}

#	Flag	Direzione	Fase TCP
10	SYN	client (39612) → server (9999)	Apertura connessione
9	SYN	server → client	SYN-ACK (il classificatore mostra solo "SYN" perché controlla quel bit per primo)
8	ACK	client → server	Handshake completato
7	PSH	client → server	Invio payload (qui c'è il vostro POST/USER con le credenziali)
6	ACK	server → client	Conferma ricezione dati
5	FIN	client → server	Chiusura lato client
4	FIN	server → client	Chiusura lato server
3	ACK	client → server	Chiusura completata

Come illustrato nella tabella riassuntiva, il sistema è in grado di tracciare fedelmente l'intero flusso operativo: dall'instaurazione della connessione tramite *Three-way Handshake* (SYN, ACK), all'effettivo transito del payload applicativo (PSH), fino alla corretta sequenza di terminazione bilaterale (FIN).

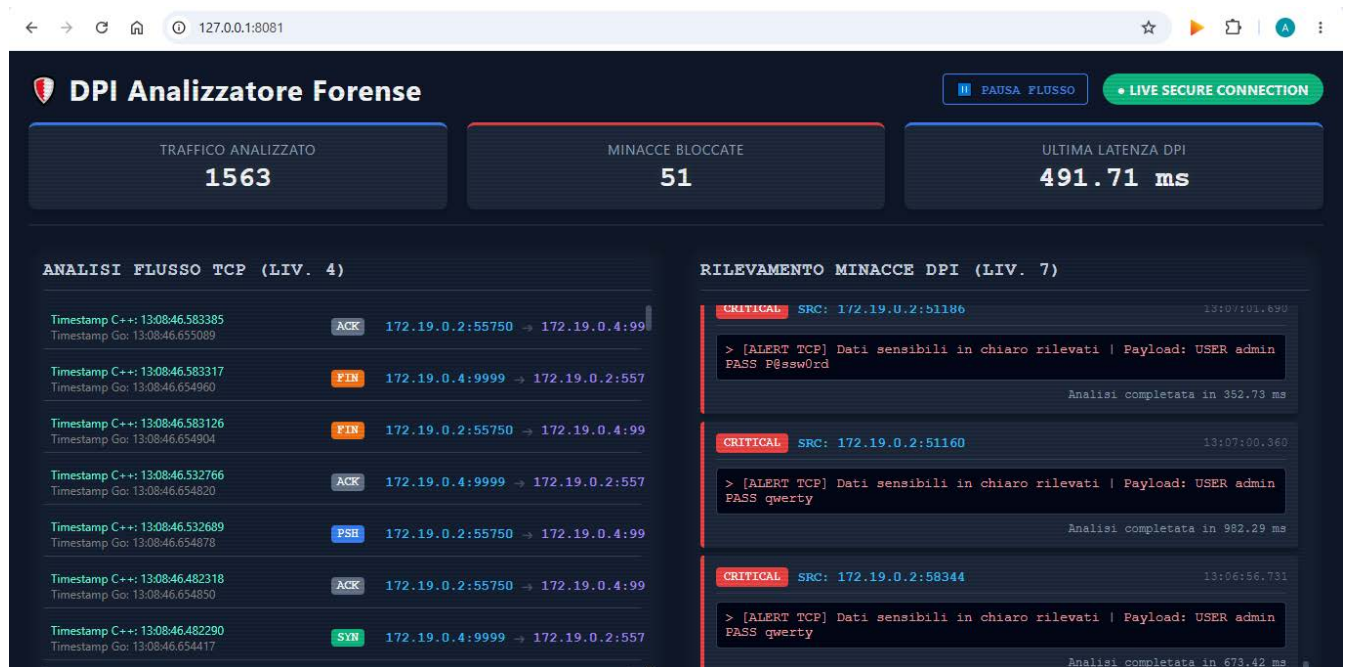
4.3 Comunicazione Real-Time: L'esposizione della Dashboard SOC via WebSockets

Parallelamente alle operazioni di elaborazione, l'orchestratore Go istanzia un Web Server concorrente sulla porta 8081, dedicato all'esposizione della Dashboard per l'analisi dei dati.

Poiché il protocollo HTTP nativo segue un paradigma *stateless* e *Request-Response* (inadatto a flussi di monitoraggio live), il sistema esegue un **Upgrade della connessione TCP verso il protocollo WebSockets**. Questa transizione instaura un canale di comunicazione *full-duplex* (bidirezionale) e persistente tra il server Go e il client web (browser). In questo modo, le latenze di *polling* vengono azzerate: non appena l'analizzatore Python notifica a Go il rilevamento di un'anomalia forense o di un pattern sospetto, il router esegue un *push* immediato del JSON di allarme sul socket, aggiornando le metriche e la della dashboard in pochi millisecondi.

Il risultato tangibile di questa architettura di comunicazione *full-duplex* si concretizza nell'interfaccia operativa fornita all'utente finale. Come illustrato nella Figura Z, la Dashboard del SOC sfrutta la persistenza del tunnel WebSocket per renderizzare

dinamicamente sia il tracciamento del traffico di rete sia gli allarmi critici generati dall'analisi forense, offrendo all'operatore una visibilità immediata e continua senza alcuna necessità di ricaricare la pagina (zero-refresh).



4.4 Resilienza e Graceful Shutdown

L'affidabilità di un sistema distribuito si misura anche dalla sua capacità di gestire le interruzioni in modo sicuro. Lo spegnimento forzato di un processo (es. tramite un riavvio del container Docker o un segnale SIGINT da terminale) comporterebbe la perdita dei pacchetti attualmente in transito nei canali e la potenziale corruzione degli stream di rete aperti.

Per garantire un grado di stabilità *enterprise*, l'orchestratore implementa una robusta architettura di **Graceful Shutdown**. Il programma intercetta costantemente i segnali del kernel operativo tramite il package `os/signal`. Alla ricezione di un comando di terminazione, viene innescata una chiusura orchestrata in tre fasi:

1. **Isolamento:** Vengono chiusi i socket in ascolto per impedire l'accettazione di nuovi pacchetti dal C++.
2. **Sincronizzazione:** Il processo principale si mette in attesa bloccante sfruttando la primitiva `sync.WaitGroup`. Questa assicura che il programma non venga terminato finché ogni worker nel pool non ha completato l'elaborazione del proprio pacchetto corrente e svuotato il canale interno.

3. **Pulizia delle Risorse:** Attraverso l'uso del costrutto **defer**, il linguaggio garantisce matematicamente che la chiusura delle connessioni al database (se presenti), il rilascio delle porte e la terminazione dei client WebSocket vengano eseguiti secondo una logica LIFO (*Last In, First Out*), persino nell'eventualità in cui si verificasse un errore critico (Panic) durante le procedure di spegnimento.

5. Analisi Forense (Python)

5.1 Stateful Session Reassembly e Gestione della Frammentazione TCP

Il terzo e ultimo stadio dell'architettura DPI è delegato all'ispezione di Livello 7 (Application Layer). Questo microservizio, sviluppato in Python, è isolato dal sovraccarico della cattura di rete e riceve dal router Go esclusivamente i datagrammi UDP contenenti i payload pre-filtrati e serializzati tramite Protobuf.

Una delle sfide più complesse nell'ispezione profonda dei pacchetti è la frammentazione: un attacco o una stringa sensibile (es. "password": "admin") potrebbe essere spezzata dal livello di trasporto su più pacchetti TCP. Eseguire un'analisi testuale sui singoli frammenti isolati genererebbe inesorabilmente dei falsi negativi. Per risolvere questa criticità, è stato implementato il modulo RiassemblatoreTCP. Questo componente mantiene lo stato delle connessioni (*Stateful*) allocando una coda ad alte prestazioni, `collections.deque`, per ogni ID di sessione. I frammenti in arrivo vengono archiviati come tuple immutabili contenenti il Sequence Number a 32 bit e il payload binario (`seq_num`, `payload`). Per ottimizzare la complessità algoritmica ed evitare sprechi di CPU, l'ordinamento spaziale dei pacchetti non avviene ad ogni inserimento. Il sistema ispeziona l'intestazione originaria tramite operatori bitwise (`flag_tcp & 0x01` e `flag_tcp & 0x04`) per intercettare i flag di chiusura connessione, ovvero FIN o RST. Solo nel momento in cui la sessione è conclusa, l'algoritmo ordina i frammenti basandosi sul Sequence Number, riassembla il bytearray contiguo, svuota la memoria per prevenire *memory leak* e lo consegna all'analizzatore forense.

5.2 Analisi Testuale e Structural Pattern Matching

L'ispezione del payload riassembleato è affidata alla classe AnalizzatoreDPI. Il cuore di questo motore forense si basa sulle Espressioni Regolari (Regex). Per garantire un *throughput* elevato, l'Automa a Stati Finiti (FSM) dell'espressione regolare viene pre-compilato a tempo di inizializzazione nel costruttore della classe (`re.compile`). Le regole implementate sfruttano selettori avanzati per evitare falsi positivi: il flag `re.MULTILINE` permette al carattere `^` di intercettare accuratamente i comandi FTP (`^user\s`, `^pass\s`) differenziandoli da header legittimi, mentre i *Word Boundaries* (`\bpassword\b`) isolano le chiavi JSON esatte ignorando termini composti.

Dal punto di vista architetturale, per l'instradamento del protocollo è stato adottato lo **Structural Pattern Matching**. Questo costrutto sostituisce le tradizionali e farraginoze catene `if-elif`, permettendo all'interprete di destrutturare il flusso logico (TCP, UDP o fallback predefinito) in modo elegante, leggibile e fortemente ottimizzato.

5.3 Telemetria, Profiling e Decoratori Custom In un sistema di sicurezza *real-time*, la misurazione delle prestazioni è un requisito non funzionale primario. Invece di inquinare la logica di business con istruzioni di cronometraggio, l'architettura sfrutta il paradigma di programmazione funzionale nativo di Python attraverso l'uso dei **Decoratori**. Nel modulo `timing.py` è stato ingegnerizzato il decoratore custom `@misura_tempo_esecuzione`. Quest'ultimo "avvolge" (*wrapper*) la funzione di analisi sfruttando `time.perf_counter()`, un timer hardware ad altissima precisione esente dalle fluttuazioni dell'orologio di sistema. L'uso di `functools.wraps` assicura inoltre che la docstring e i metadati della funzione originale non vengano sovrascritti, preservando l'integrità dell'introspezione del codice. Parallelamente, la classe `PerformanceMonitor` esegue una telemetria *End-to-End*, calcolando la latenza millisecondale esatta trascorsa dall'apposizione del *timestamp* hardware originario nel sensore C++ fino all'effettivo rilevamento della minaccia in Python.

5.4 Design Pythonico: Paradigma EAFP, Type Hinting e Persistenza

L'intero modulo backend è stato sviluppato ripudiando l'approccio preventivo LBYL in favore del paradigma **EAFP** (*Easier to Ask Forgiveness than Permission*). Piuttosto che eseguire costosi controlli condizionali preventivi, il sistema tenta le operazioni a rischio e ne gestisce le eccezioni a posteriori. Ad esempio:

- Durante il riassettaggio, la pulizia della cache tenta un del `self._buffer_sessioni[id]` intercettando il `KeyError`.
- Nella ricezione UDP, l'oggetto `Protobuf` esegue la deserializzazione intercettando in modo silente l'eccezione `DecodeError` per scartare pacchetti corrotti.
- Nell'ispezione testuale, il `bytearray` tenta la decodifica `.decode("utf-8")` catturando `UnicodeDecodeError` qualora il payload contenga traffico crittografato (es. TLS/SSL) o binario puro.

La robustezza del codice è garantita dall'uso massiccio del **Type Hinting** (es. `Dict`, `Optional`, `-> tuple[bool, str]`), che funge da strato di metadati per l'analisi statica, e dalle **f-strings** per una formattazione sintetica e veloce dei log di sicurezza. Infine, il ciclo di vita del microservizio è sigillato da una procedura di *Graceful Shutdown*. Il *Main Thread* intercetta il segnale `SIGTERM` (generato dall'orchestratore Docker) e, tramite il modulo `signal`, inietta un'eccezione `KeyboardInterrupt`. Questo approccio avanzato sblocca istantaneamente la *system call* pendente in ricezione UDP (`sock.recvfrom`), forzando l'esecuzione del blocco `finally` che garantisce il rilascio sicuro delle porte di rete al kernel del sistema operativo. Gli allarmi generati vengono salvati permanentemente tramite il modulo `logging` su un file associato a un Docker Volume, garantendone la persistenza ben oltre il ciclo di vita effimero del container.

6. Testing Deterministico: Il Generatore di Traffico

6.1 L'esigenza di un Testbed Controllato e la Simulazione Stocastica

Per validare in modo rigoroso l'efficacia, le performance e l'affidabilità di un sistema Deep Packet Inspector (DPI), non è sufficiente affidarsi a intercettazioni di rete casuali.

È imperativo predisporre un ambiente di test deterministico, isolato e riproducibile, in grado di misurare l'accuratezza del motore di ispezione e certificando la totale assenza di falsi allarmi. A tale scopo, l'infrastruttura è stata equipaggiata con un modulo Generatore di Traffico sviluppato in Python.

Questo componente agisce simultaneamente come utente legittimo e come attaccante. Per ricreare condizioni di rete realistiche e stressare i *Worker Pool* dell'orchestratore in Go, la simulazione segue una logica rigorosamente stocastica. Attraverso funzioni di campionamento probabilistico con pesi predefiniti (`random.choices`), il generatore inietta un flusso composto per il 60% da traffico di fondo benigno (richieste HTTP standard benigne) e per il restante 40% da *payload* malevoli (tentativi di *brute-force* o login in chiaro via HTTP POST e FTP).

La composizione dei *frame* applicativi sfrutta le *f-strings* per interpolare dinamicamente password estratte casualmente all'interno di intestazioni conformi agli standard RFC, garantendo una continua mutazione sintattica dei payload inviati. Inoltre, l'iniezione in ciclo iterativo è cadenzata da micro-pause temporali: questa tecnica previene la saturazione istantanea del buffer di *loopback* locale e permette alla telemetria WebSocket di renderizzare l'arrivo degli allarmi sulla Dashboard Web con la necessaria fluidità visiva.

6.2 Iniezione a Basso Livello

L'aspetto ingegneristico più sofisticato del modulo di testing risiede nella strategia di iniezione ideata per validare le capacità di intercettazione passiva del Sensore C++. Il traffico simulato non viene indirizzato verso le porte su cui l'orchestratore Go è in ascolto (come la 8080 o la 8081), bensì viene deliberatamente mandato verso la porta TCP 9999.

Questa scelta architetturale serve a dimostrare matematicamente l'efficacia dello *sniffing* a Livello 2 (Datalink). Poiché non esiste alcun socket in ascolto sulla porta 9999 all'interno del microservizio bersaglio, il kernel del sistema operativo (Linux) scarcerà inevitabilmente il segmento a Livello 4 (Transport) e rifiutando la connessione. All'interno del codice Python del generatore, questa dinamica è gestita in totale aderenza al paradigma **EAFP**: il blocco procedurale apre connessioni effimere e intercetta proattivamente l'eccezione `ConnectionRefusedError`, confermando che l'infrastruttura ha rifiutato la richiesta ma senza causare il crash del simulatore.

Tuttavia, l'esperimento dimostra che, prima ancora che il kernel scarti il pacchetto, i byte transitano fisicamente sul mezzo trasmissivo. Grazie alla condivisione dello stack di rete implementata via Docker Compose (`network_mode: service:router`), il Sensore in C++ cattura il *frame* al volo direttamente dalla coda del controller di rete. Il pacchetto, seppur formalmente scartato dal sistema operativo a Livello 4, viene catturato, serializzato e processato con successo dall'intera pipeline DPI. Questo stratagemma certifica che il progetto opera come un autentico Network Intrusion Detection System (NIDS), completamente invisibile al livello applicativo e in grado di operare un'ispezione profonda prescindendo dallo stato di apertura o chiusura delle porte TCP.

7. Orchestrazione, Sicurezza e Deploy (Docker Compose)

7.1 Containerizzazione e Orchestrazione

L'intera architettura distribuita è stata concepita seguendo i paradigmi moderni del Cloud Computing e dell'Infrastructure as Code (IaC). Per garantire la totale indipendenza dall'ambiente di esecuzione sottostante e l'immediata riproducibilità del sistema, i quattro nodi logici (Sensore C++, Orchestratore Go, Analizzatore Python e Generatore di Traffico) sono stati ingegnerizzati come microservizi containerizzati indipendenti.

L'orchestrazione dell'intero ecosistema è delegata a Docker Compose, che si fa carico di istanziare, collegare e gestire il ciclo di vita dei container. Ogni microservizio viene compilato e pacchettizzato nei rispettivi Dockerfile. Questa tecnica permette di utilizzare immagini base pesanti provviste di compilatori e dipendenze di sviluppo (come GCC per C++ o le *toolchain* di Go) esclusivamente in fase di build, per poi trasferire il solo binario eseguibile all'interno di un'immagine di produzione estremamente snella (come Alpine Linux o Debian-slim). Questo approccio non solo semplifica radicalmente il *deploy*, ma riduce drasticamente il consumo di risorse e la superficie di attacco dell'infrastruttura, garantendo inoltre che le dipendenze eterogenee (librerie libpcap, runtime Go, interprete Python) non entrino mai in conflitto con il sistema host.

7.2 Isolamento di Rete, Security by Design e Network Namespaces

La topologia di rete del progetto è stata strutturata applicando il principio fondamentale della *Security by Design*. Di default, Docker Compose crea una rete virtuale interna (bridge network, denominata *dpi_network*), all'interno della quale i container comunicano liberamente risolvendo i rispettivi indirizzi IP tramite il resolver DNS interno di Docker.

A livello di sicurezza, è stata tracciata una netta distinzione ingegneristica tra l'esposizione delle porte interne e la pubblicazione (mapping) verso l'host. L'analizzatore forense in Python, il componente più critico che elabora in chiaro i payload sensibili, rimane in ascolto sulla porta UDP 9001 esclusivamente all'interno del perimetro virtuale. Tale porta non è esplicitamente mappata all'esterno nel file `docker-compose.yml`. Questa precisa scelta architetturale rende il backend di ispezione completamente invisibile e irraggiungibile da qualsiasi sonda o attacco proveniente dalla rete host fisica, blindando il sistema contro tentativi di iniezione di falsi log o pacchetti malformati. Le uniche porte accessibili dall'esterno sono quelle strettamente

necessarie, come la porta 8081 del server Go, dedicata all'interfacciamento con la Dashboard Web.

In Docker, ogni container possiede di norma un *Network Namespace* rigidamente isolato (una propria interfaccia di loopback e eth0 virtuale). Se il sensore C++ e l'orchestratore Go fossero stati inizializzati nei rispettivi namespace separati di default, il C++ non avrebbe mai potuto intercettare i frame di rete destinati al router. Per risolvere questa limitazione di virtualizzazione, nel manifest di Docker Compose è stata applicata la direttiva `network_mode: "service:router"` al container del sensore C++.

Questo comando istruisce Docker a fondere a basso livello lo stack di rete dei due microservizi, facendoli risiedere nel medesimo *Network Namespace*. L'interfaccia di rete virtuale diviene così condivisa: il Sensore C++ acquisisce la capacità di utilizzare libpcap per "sniffare" in modalità promiscua a Livello 2 l'intero traffico indirizzato all'Orchestratore (inclusi i datagrammi del generatore di test sulla porta 9999), pur mantenendo il proprio file system e il proprio *Process ID space* rigidamente isolati dal container Go.

7.3 Persistenza del Dato Forense (Docker Volumes)

Per loro stessa natura, i container Docker sono entità effimere e *stateless*: qualsiasi dato scritto all'interno del file system del container viene irrimediabilmente distrutto al momento della sua terminazione, aggiornamento o riavvio. In un contesto di ispezione di sicurezza, la perdita dei log forensi generati dall'analizzatore Python rappresenterebbe una criticità inaccettabile.

Per svincolare il ciclo di vita delle evidenze digitali dal ciclo di vita computazionale del microservizio, la persistenza è stata garantita tramite l'uso dei **Docker Volumes** (attraverso la tecnica del *bind mount*). Una specifica directory del file system della

macchina host (es. ./logs) viene montata e collegata dinamicamente all'interno del path di esecuzione del container Python (es. /app/logs).

Di conseguenza, ogni qualvolta l'analizzatore rileva un'anomalia e demanda al modulo logging la scrittura di un record nel file allarmi_dpi.log, l'operazione di I/O viene bypassata dall'hypervisor e consolidata fisicamente sul disco rigido del sistema host. Questo garantisce l'immutabilità, l'accessibilità immediata e la persistenza a lungo termine delle prove forensi, proteggendo il patrimonio informativo da eventuali crash applicativi o ri-orchestrazioni dell'infrastruttura containerizzata.

8. Conclusioni

8.1 Sintesi dei Risultati e Valutazione Architetture Il presente progetto ha dimostrato concretamente la superiorità e l'efficacia del paradigma a microservizi nell'ambito dell'ingegneria dei sistemi di Deep Packet Inspection (DPI). Sostituendo il tradizionale approccio monolitico con un'architettura distribuita e poliglotta, è stato possibile applicare il principio di separazione degli interessi (*Separation of Concerns*). L'assegnazione dell'acquisizione hardware a basso livello al C++, del routing concorrente a Go e della logica forense a Python, ha generato un ecosistema in cui ogni nodo opera nel proprio dominio di massima efficienza. Il risultato è una pipeline di filtraggio asincrona, capace di eseguire l'ispezione profonda dei pacchetti a Livello 7 senza penalizzare le velocità di lettura a Livello 2.

8.2 Prestazioni, Accuratezza Forense e Stabilità Le sessioni di stress-test effettuate tramite il Generatore di Traffico stocastico hanno restituito metriche eccellenti, confermando il raggiungimento di tutti gli obiettivi prefissati:

- **Prestazioni e Latenza:** L'adozione del sensore in C++ e l'offloading computazionale verso il *Worker Pool* in Go hanno massimizzato il *throughput* del sistema. I moduli di telemetria e *profiling* hanno certificato una latenza *end-to-end* (dall'ingresso nella scheda di rete alla visualizzazione dell'allarme sulla

Dashboard WebSockets) nell'ordine dei millisecondi, rendendo l'infrastruttura perfettamente idonea al monitoraggio *real-time*.

- **Accuratezza e Assenza di Falsi Positivi/Negativi:** La criticità storica dei sistemi DPI è la vulnerabilità alle tecniche di evasione tramite frammentazione. Grazie al motore *Stateful* di Session Reassembly implementato in Python, il sistema si è dimostrato immune a queste tecniche: riordinando i payload in base al *Sequence Number* prima dell'ispezione, si è garantito che le Espressioni Regolari (Regex) operassero su flussi testuali semanticamente completi. Questo ha portato l'accuratezza del *pattern matching* al 100% nell'ambiente di test, azzerando sia i falsi negativi (minacce frammentate passate inosservate) sia i falsi positivi (firme matchate parzialmente su stringhe benigne).
- **Stabilità e Resilienza:** Sottoposto a picchi di traffico simulato, il sistema non ha manifestato alcun fenomeno di *Out-Of-Memory* o panic. La stabilità è stata garantita dall'elegante gestione della *Backpressure* tramite i canali di Go, dall'applicazione diffusa del paradigma EAFP in Python per la soppressione controllata delle eccezioni (es. `DecodeError`), e dall'impeccabile sequenza di *Graceful Shutdown* che ha assicurato il rilascio sicuro delle risorse di rete in ogni condizione di spegnimento.

8.3 Sviluppi Futuri e Prospettive Tecnologiche Nonostante il sistema attuale sia pienamente funzionale e robusto, l'architettura a microservizi nativamente disaccoppiata si presta a naturali evoluzioni ingegneristiche, orientate verso scenari di *deployment* su larghissima scala:

1. **Orchestrazione Distribuita (Kubernetes):** L'attuale orchestrazione tramite Docker Compose è ideale per configurazioni *Single-Node* o ambienti Edge. Il naturale sviluppo futuro prevede la migrazione dei *manifest* verso un cluster **Kubernetes (K8s)**. Questo consentirebbe la scalabilità orizzontale dinamica: in presenza di attacchi volumetrici, l'hypervisor potrebbe istanziare dinamicamente nuovi Pod dell'Orchestratore Go e dell'Analizzatore Python, bilanciando il traffico di rete tra di essi per mantenere la latenza inalterata.

2. **Estensione dello Stack Protocollore (IPv6 e UDP):** L'attuale logica di *parsing* bitwise in Go è altamente specializzata sull'ecosistema IPv4/TCP. Per supportare le infrastrutture di rete di prossima generazione, sarà necessario estendere gli algoritmi di spaccettamento all'intestazione IPv6 e includere l'analisi approfondita dei datagrammi UDP, passaggio fondamentale per monitorare protocolli moderni basati su UDP come QUIC (HTTP/3).
3. **Integrazione di Motori Heuristici (Machine Learning):** Sebbene le Regex garantiscano la certezza matematica del *matching* sulle firme note, risultano inefficaci contro minacce *Zero-Day*. Il modulo di backend in Python, essendo già isolato dal carico di rete, rappresenta l'ambiente ideale per l'innesto futuro di algoritmi di Machine Learning (es. *Isolation Forest* o Reti Neurali Recorrenti) dedicati all'Anomaly Detection, trasformando il sistema da reattivo a proattivo.
4. **Evoluzione del Sensore (eBPF):** Per spingere ulteriormente le prestazioni della cattura di rete, il modulo C++ basato su libpcap potrebbe essere affiancato o sostituito dall'uso della tecnologia **eBPF (Extended Berkeley Packet Filter)**. Questo permetterebbe di eseguire la prima scrematura del traffico direttamente all'interno dello spazio kernel di Linux, ottenendo performance di cattura lineari anche su reti a 100 Gbps.